



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

MIGUEL CHEN

BSc in Electrical and Computer Engineering

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS PREPARATION REPORT

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS PREPARATION REPORT

MIGUEL CHEN

BSc in Electrical and Computer Engineering

Supervisor: Pedro Miguel Figueiredo Amaral
Full Professor, NOVA University Lisbon

Examination Committee

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026

Adversarial Attacks On DRL Based Network Routing Systems

Master's Thesis Preparation Report

Copyright © Miguel Chen, NOVA School of School and Technology, NOVA University Lisbon.

The NOVA School of School and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

This document was produced with the pdfL^AT_EX [1] typesetting system and the NOVAthesis template (v7.9.1) [2].

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29).
- [2] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourengo/novathesis/raw/main/template.pdf>.

DISCLOSURE OF DELEGATION TO GENERATIVE ARTIFICIAL INTELLIGENCE

Miguel Chen declares¹ the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [3], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- | | | |
|--|--|---|
| ☐ Idea generation | ☐ Formulating research questions and hypotheses | ☐ Feasibility assessment and risk evaluation |
| ☐ Defining the research objective | | ☐ Preliminary hypothesis testing |

Literature Review

- | | | |
|---|--|---|
| ☒ Literature search and systematization | ☐ Writing the literature review | ☐ Evaluation of novelty and identification of gaps |
| | ☐ Analysis of market trends and/or patent environment | |

Methodology

- | | | |
|--|--|--|
| ☐ Research design | ☐ Development of experimental or research protocols | ☐ Selection of research methods |
|--|--|--|

Software Development and Automation

- | | | |
|--|---|---|
| ☐ Code generation | ☐ Process automation | ☐ Creation of algorithms for data analysis |
| ☐ Code optimization | | |

Data Management

- | | | |
|--|---|--|
| ☐ Data collection | ☐ Data curation and organization | ☐ Reproducibility testing |
| ☐ Validation | ☐ Data analysis | |
| ☐ Data cleaning | ☐ Visualization | |

¹This declaration was generated using pdfL^AT_EX [1] and the L^AT_EX package aidisclose [2].

Writing and Editing

- | | | |
|--|--|--|
| ☒ Text generation | ☐ Formulation of conclusions | ☐ Translation |
| ☒ Proofreading and editing | ☐ Adapting and adjusting emotional tone | ☐ Reformatting |
| ☒ Summarizing text | | ☐ Press releases and outreach materials |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Supervision

- | | | |
|---|--|--|
| ☐ Quality assessment | ☐ Identification of limitations | ☐ Publication support |
| ☐ Trend identification | ☐ Recommendations | |

Generative AI tools used: Perplexity 12.12.25.

CONTENTS

Disclosure of Delegation to Generative Artificial Intelligence	iii
Contents	v
List of Figures	vii
List of Tables	ix
List of Algorithms	xi
1 Introduction	1
2 Literature review	3
2.1 DRL definition and overview	3
2.2 Deep Q-Networks	4
2.2.1 Duelling Q-networks	5
2.3 MADDPG	6
2.3.1 MADDPG in Network Routing	7
2.4 Taxonomy on Adversarial attacks on MADDPG	8
2.4.1 Observation Space Attacks	8
2.4.2 Action-Space Attacks	9
2.4.3 Communication-Space Attacks	10
2.4.4 Poisoning and Backdoor Attacks	10
2.4.5 Transferability and Black-Box Attacks	11
2.4.6 FGSM	11
2.4.7 PGD	12
2.4.8 Overview of adversarial attacks	14
2.5 Graph Neural Networks	15
2.5.1 GNN applications in network routing	17
2.6 Taxonomy on Adversarial attacks on GNNs	18
2.6.1 Attacker knowledge	18
2.6.2 Attack Timing	19
2.6.3 Attack Strategy (Perturbation Goal)	19

2.6.4	Attack Manipulation	19
2.6.5	Attack Specificity	20
2.6.6	Task level	20
2.6.7	Examples and characteristics of adversarial attacks on GNNs . .	20
2.6.8	Transferability-Focused Attacks	21
3	Methodology	23
3.1	Starting Point and Scope of This Work	23
3.1.1	The routing system under test	23
3.1.2	The FGSM baseline	24
3.1.3	Contribution of this thesis	24
3.2	Victim Architectures Under Test	25
3.2.1	Central (CC) versus local (LC) critic	25
3.2.2	Simple versus duelling value head	25
3.2.3	With and without GNN encoding	26
3.2.4	Variant matrix	26
3.3	PGD-Based Adversarial Attacks on MADDPG Routing Agents	27
3.3.1	Threat model and admissible perturbation set	27
3.3.2	(A) Coordinated multi-agent perturbation	28
3.3.3	(B) Strategically-timed attacks under an L_0 budget	30
3.3.4	Evaluation protocol	31
4	Statistics and Results	33
	Bibliography	35

LIST OF FIGURES

2.1	Diagram of Adversarial Attack characteristics on Graph Neural Networks .	18
-----	--	----

LIST OF TABLES

2.1	Taxonomy of Adversarial Attacks on MADDPG	14
2.2	Taxonomy of Adversarial Attacks on MADDPG part 2	15
2.3	Attacker’s Knowledge (Information Access Level)	19
2.4	Summary of Adversarial Attacks on GNNs	20
3.1	MADDPG victim variants under test, named <critic domain>-<value head>[-GNN].	26

LIST OF ALGORITHMS

1	Coordinated multi-agent PGD (one time step)	30
2	Critical-state timing gate	31

INTRODUCTION

As the field of telecommunications progresses issues such as scalability, routing, quality of service, resource management and network security become increasingly complex. Recent studies turn to Artificial Intelligence (AI) and Machine Learning (ML) for solutions, specifically MADDPG and GNN models involving DRL agents to make optimal routing decisions.

The convergence of Deep Reinforcement Learning (DRL), Graph Neural Networks (GNNs), and automatization of network management and configuration represents a radical change in how modern networks are controlled and optimized. Traditional routing protocols, such as OSPF and BGP, employ static rule-based approaches that struggle with dynamic network environments characterized by heterogeneous Quality-of-Service (QoS) requirements, variable traffic patterns, and multiple optimization constraints. Deep Reinforcement Learning has emerged as a powerful alternative, enabling routing agents to learn adaptive policies directly from real-time network observations without explicit programming of optimization rules. [4]

Multi-Agent DRL (MADRL), more specifically utilizing Multi-Agent Deep Deterministic Policy Gradient (MADDPG), systems extend this capability to distributed network environments where multiple autonomous agents must coordinate routing decisions across different nodes while maintaining local operational autonomy. When enhanced with Graph Neural Networks that naturally exploit the graph-structured topology of networks, these systems achieve improved scalability, topology-awareness, and the ability to generalize across different network configurations. [5] [6] [7] [8] [9].

However DRL models have many well documented security vulnerabilities, consequences of which remain underexplored in the context of network routing. Common attacks such as the fast gradient sign method (FGSM) [10], the projected gradient descent (PGD) [11] and the Carlini and Wagner (CW) attack [12] can manipulate model inputs to degrade performance or even implant backdoors as a gateway to further exploits [13]. Some research has been done on the effects of adversarial attacks on DRL models in the context of image classification, mostly focusing on model robustness versus adversarial examples. [14] [10] [15].

This thesis aims to explore the effects of adversarial attacks on MADDPG and GNN models in the context of network routing. Specifically, we will investigate how these attacks impact the performance of these models when used for routing decisions in a simulated network environment. We will implement and adapt FGSM and PGD algorithms to the network context, evaluating their effectiveness in degrading network performance metrics across diverse scenarios.

The main contributions of this thesis are as follows:

- Adapting and validating FGSM and PGD adversarial attack algorithms to the context of network routing.
- Evaluating the impact of these attacks on the performance of MADDPG and GNN routing agents across various network topologies and traffic patterns.

The structure of this thesis preparation report is as follows: Chapter 1 provides an introduction to the research topic, outlining the motivation and objectives of the study. Chapter 2 reviews relevant literature, summarizing existing research on MADDPG, GNNs and adversarial attacks of respective architectures. Chapter 3 details the methodology and implementation of the working environment, FGSM and PGD adapted to the network environment.

LITERATURE REVIEW

This chapter reviews relevant literature on Multi-Agent Deep Deterministic Policy Gradient (MADDPG) and Graph Neural Networks (GNNs), We will explore their architectures, the proposed applications in network routing and current literature on their vulnerabilities to adversarial attacks.

2.1 DRL definition and overview

Deep Reinforcement Learning (DRL) is a subfield of machine learning that combines reinforcement learning (RL) principles with deep neural networks to enable agents to learn optimal policies in complex environments. In DRL, an agent interacts with an environment by taking actions based on its current state, receiving feedback in the form of rewards or penalties. The goal of the agent is to learn a policy that maximizes the cumulative reward over time. Unlike traditional rule-based approaches, DRL agents learn directly from state observations, making them particularly suited for network optimization tasks where hand-crafted rules cannot capture the full complexity of system dynamics. [16] [17] [18] In the context of network routing, DRL enables agents to discover adaptive forwarding policies that respond to real-time network conditions (congestion, link failures, traffic demands) without requiring explicit model of the environment. [19] [20] [21] The environment is naturally modeled as a Markov Decision Process (MDP) or Partially Observable MDP (POMDP) where states encode link loads and topology information, actions correspond to next-hop or path selection decisions, and rewards capture delay, throughput, or composite QoS metrics. [22]

However, DRL models are vulnerable to adversarial attacks, where malicious actors can input adversarial examples designed to mislead the model into making incorrect decisions [23], these are often called either evasion attacks when introduced during test-time or poisoning attacks when introduced during training. Lin et al. [24] further introduce ‘enchanted attacks’, which craft a sequence of adversarial perturbations to drive the agent toward specific target states. [25] [13] These attacks exploit the inherent lack of robustness in the model, leading to suboptimal decision-making in many application areas. [26] [27] [28] Common adversarial attack methods include the Fast Gradient Sign Method (FGSM),

Projected Gradient Descent (PGD), the Carlini and Wagner (CW) attack and more recently, the distribution-Aware Projected Gradient Descent (DAPGD). [15] [29]

2.2 Deep Q-Networks

Deep Q-Networks integrate deep convolutional neural networks with Q-learning, a reinforcement learning algorithm, to learn optimal action-selection policies directly from high-dimensional sensory inputs without manual feature engineering. [30] [31] The architecture parameterizes the action-value function, the Q-function, using a deep neural network whose input consists of preprocessed high-dimensional observations (e.g., raw pixel data) and whose output comprises value estimates for each possible action, representing the expected cumulative future reward associated with each action selection. Training stability, historically a critical challenge when combining neural networks with temporal-difference learning, is achieved through two key mechanisms: experience replay, a technique wherein the network is trained on minibatches of randomly sampled experiences drawn from a memory buffer, thereby breaking temporal correlations in the observation sequence and smoothing shifts in the data distribution. [30] [31] The network is trained via stochastic gradient descent to minimize the temporal-difference error, defined by the squared Bellman residual between the predicted Q-value and the target value computed as the immediate reward plus the discounted maximum Q-value of the subsequent state as estimated by the target network. [30]

$$L_i(\theta_i) = \mathbb{E}_{s,a \sim \rho(\cdot)} [(y_i - Q(s, a; \theta_i))^2] \quad (2.1)$$

where the target value is:

$$y_i = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \middle| s, a \right] \quad (2.2)$$

- $L_i(\theta_i)$: Loss function at iteration i with network parameters θ_i
- θ_i : Network parameters (weights and biases) at iteration i
- θ_{i-1} : Network parameters from the previous iteration $i - 1$ (held fixed during optimization of L_i)
- $\mathbb{E}_{s,a \sim \rho(\cdot)}$: Expectation over state-action pairs (s, a) sampled from the behaviour distribution $\rho(s, a)$
- y_i : Target value (Bellman target) for iteration i
- $Q(s, a; \theta_i)$: Predicted action-value function (Q-function) that estimates the expected cumulative discounted reward for taking action a in state s using network parameters θ_i

- r : Immediate scalar reward received from the environment after taking action a in state s
- γ : Discount factor; a scalar in $[0, 1]$ that weights the importance of future rewards relative to immediate rewards
- $\max_{a'} Q(s', a'; \theta_{i-1})$: Maximum Q-value across all possible actions a' in the successor state s' , computed using the previous iteration's parameters θ_{i-1}
- s : Current state, represented as a sequence of preprocessed observations (e.g., stacked pixel frames)
- a : Action taken in state s ; chosen from the discrete action set
- s' : Successor state resulting from taking action a in state s
- $\mathcal{E}$: The environment (in the DQN paper: the Atari 2600 emulator [30]); transition dynamics are unknown and learned implicitly
- $\rho(s, a)$: Behaviour distribution; the probability distribution of state-action pairs encountered during training (typically from an ϵ -greedy policy)

[30]

DQN has been successfully applied to network routing tasks, where the agent learns to make optimal forwarding decisions based on various real-time network state observations. A recent paper by Xuan Geng et al. (2025) [32] Proposes a DQN-based routing protocol that optimizes both path selection and energy efficiency thereby significantly extending network lifetime of underwater acoustic sensor networks (UASNs). DQL has also been proposed as a solution to help Software Defined Networks (SDN) based data-centers dealing with "mice" flows, which are short-lived flows with relatively small data sizes that can congest the network if not handled properly and "elephant" flows, that are long lived and carry large amounts of data. 2 DQNs are built, one for each type of flow, that train Convolutional Neural Networks (CNN) to optimize routing decisions. [33]

2.2.1 Duelling Q-networks

Dueling Network Architectures for Deep Reinforcement Learning represent a structural innovation to the standard Deep Q-Network architecture in which the action-value function is decomposed into two independent, streams that share a common feature representation before final aggregation [34, 35]. Specifically, the architecture bifurcates into a value stream $V(s; \theta_v)$ that estimates the scalar value of a given state independent of action selection, and an advantage stream $A(s, a; \theta_a)$ that computes the relative advantage of executing a particular action within that state, with both streams processing shared convolutional features derived from the input observation [34]. The final action-value estimate is obtained through a dueling layer that performs a learned aggregation of these

two quantities and provides a normalization mechanism that stabilizes gradient-based learning and enforces identifiability [34]. This architectural separation facilitates more efficient learning in domains where numerous actions yield comparatively similar expected returns, by enabling the value and advantage streams to specialize in capturing specific aspects of the policy structure while maintaining a shared feature space, thereby improving sample efficiency and convergence properties [34].

In a recent paper by Zhang et al. (2022) [36] a deep dueling Q-network based routing protocol helps optimize Routing and Spectrum Allocation (RSA) in Elastic Optical Networks (EONs). The proposed method leverages the dueling architecture to effectively learn efficient resource allocation policies under dynamic traffic conditions, where connection requests arrive and terminate unpredictably. The decomposition of the Q-function into value and advantage streams allows the model to better judge the relative merits of different routing actions based on the overall network congestion and fragmentation state thereby circumventing the learning inefficiency that arises when traditional DQN attempts to differentiate between numerous similar-valued routing actions across diverse network configurations.[36]

2.3 MADDPG

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) is a deep reinforcement learning algorithm designed for multi-agent environments that extends actor-critic methods through a centralized training with decentralized execution framework, where each agent maintains a centralized action-value function conditioned on agent actions during training while using only local observations at execution time. To relax the assumption of knowing other agent's policies, MADDPG enables each agent to maintain an approximate policy of other agents by maximizing the log-probability of observed actions with entropy regularization, allowing subject agents to learn target agents policies before adapting accordingly. For enhanced robustness in competitive settings, MADDPG trains a collection of different sub-policies per agent, randomly selecting one sub-policy per episode and maintaining separate replay buffers for each sub-policy , which maximizes the ensemble objective such that agents develop diverse behavioral strategies that prevent overfitting to specific opponent policies. [37]

However, this very architecture creates exploitable attack surfaces. The centralized critic's dependence on gradient computations makes MADDPG amenable to gradient-based adversarial perturbations, while the decentralized execution phase means each agent operates independently at test time, creating coordination vulnerabilities when one or more agents receive adversarial inputs. [22, 38, 39]

2.3.1 MADDPG in Network Routing

A recent paper by Tian et al. (2023) [4] proposes a MADDPG-based approach for solving the network slicing problem in 5G networks, a resource allocation problem where diverse service types with heterogeneous QoS requirements may, at any point, change their resource demands. The proposed method, MADDPG-SR, treats each virtual network function (VNF) as an agent that learns to allocate resources and migrate dynamically with the end goal of maintaining QoS requirements in every slice. The results show the proposed method outperforms non-MADDPG implementations. [4]

A recent paper by Huang et al. (2025) [6] proposes ERA-MADDPG as an elastic routing scheme for software-defined networks (SDNs) that casts routing optimization as a cooperative multi-agent decision-making problem and uses Multi-Agent Deep Deterministic Policy Gradient (MADDPG) as its learning backbone. In this framework, network entities (e.g., switches or abstract routing agents) learn deterministic policies that select paths or adjust forwarding behavior based on observed traffic and topology information, while a centralized critic exploits global state during training to stabilize learning in the inherently non-stationary multi-agent environment. The resulting actors are then deployed in a decentralized fashion, allowing each agent to make fast local decisions that collectively adapt the end-to-end routes to time-varying traffic conditions and QoS demands. By leveraging MADDPG, ERA-MADDPG overcomes key limitations of static routing and single-agent RL methods, notably poor scalability and weak coordination, enabling continuous “elastic” adjustment of routes to jointly reduce delay, packet loss, and congestion while improving load balance. This method outperforms other DRL based algorithms on training efficiency and recovery in dynamic network topologies. [6]

Another recent design, RS-MADDPG, [40] enables each node to learn a deterministic policy that maps local observations to continuous routing decisions (e.g., flow splitting ratios or next-hop selections) while implicitly accounting for the actions of other agents, thereby mitigating the non-stationarity that plagues independent RL approaches. By leveraging MADDPG’s capacity for continuous action spaces and multi-agent credit assignment, RS-MADDPG dynamically adjusts routes in response to real-time traffic patterns, link failures, and varying QoS demands without requiring manual reconfiguration. The algorithm’s effectiveness is further enhanced by its ability to converge approximately 30 training cycles earlier than baseline methods, demonstrating that the Centralized Training Decentralized execution (CTDE) paradigm, characteristic of MADDPG, accelerates learning of cooperative policies that jointly minimize latency, balance loads, and maximize throughput. Consequently, RS-MADDPG not only outperforms conventional protocols in key performance metrics but also provides a scalable, adaptive solution for delivering differentiated QoS services in modern SDN environments. [40]

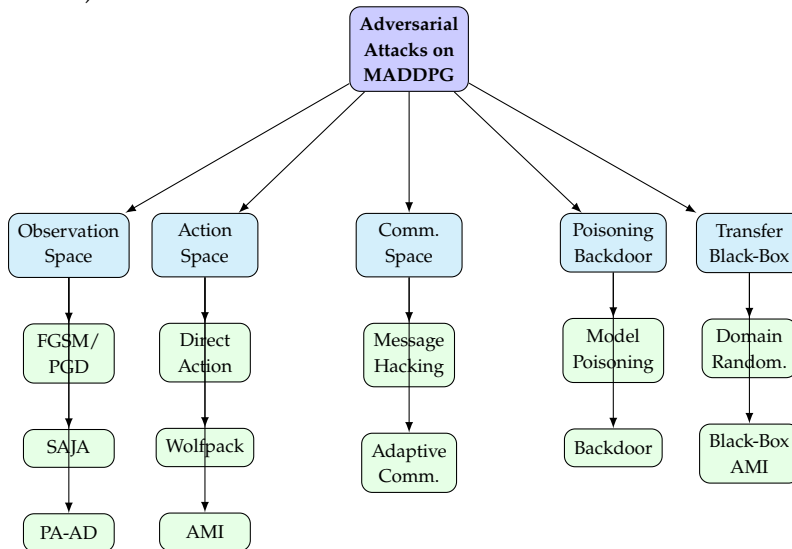
An interesting development based on the MADDPG algorithm is the Dynamic Packet Routing Algorithm using Multi-Agent Reinforcement Learning with Emergent Communication (DRAMA) [8]. DRAMA utilizes the Centralized Training Decentralized Execution

(CTDE) framework of MADDPG, but redesign the value-function architecture and information flow to overcome MADDPG’s key limitations in packet routing. While MADDPG uses a Type-II critic that takes explicit joint observation, action pairs as input, requiring fixed, enumerated discrete actions and becoming brittle when the action set changes, DRAMA replaces this with a Q-network score layer that operates on topology-aware latent features extracted by an observation encoder and a graph-based emergent communication layer. This preserves the CTDE advantage of learning from joint observations and rewards while decoupling the Q-function from a fixed action index space, so the same trained network can seamlessly score newly appearing neighbors or cope with link/router failures without the need for extensive retraining. Empirically, this MADDPG-based CTDE training combined with DRAMA’s communication and Q-network design yields comparatively better delivery rates and lower end-to-end latency than regular MADDPG implementations across a range of traffic loads and under dynamic topologies, demonstrating that DRAMA leverages the strengths of MADDPG’s training paradigm while addressing its natural difficulty with scalable, adaptive routing. [8]

Although the usage of MADDPG in network routing has increased in recent years, there is lack of research on the effects of adversarial attacks in this context.

2.4 Taxonomy on Adversarial attacks on MADDPG

Adversarial attacks against MADDPG can be systematically categorized along three dimensions: (1) perturbation domain (state, action, communication, reward, model), (2) attack timing (training-time vs. test-time), and (3) attacker knowledge (white-box, gray-box, black-box).



2.4.1 Observation Space Attacks

State-space attacks perturb the agent’s observations before the actor network processes them. These attacks exploit the gradient-based learning mechanisms underlying neural

network policies.

Gradient-Based State Perturbations: The foundational approach applies the Fast Gradient Sign Method (FGSM) or Projected Gradient Descent (PGD) to compute adversarial observations. FGSM performs a single-step perturbation in the gradient direction of the loss function,[10] while PGD extends this through iterative optimization with projection [11]. However, according to Guo et al.[41] standard FGSM attacks on MADDPG agents show limited effectiveness because the critic’s Q-value gradients alone provide unreliable guidance, imperfect critic training leads to noisy gradients. [42]

State-Action Joint Attacks (SAJA): Recent advances recognize that state and action perturbations interact synergistically. The SAJA framework operates in two phases: (1) a multi-step gradient ascent on actor and critic networks to compute adversarial states, then (2) a second gradient ascent using the perturbed state to craft adversarial actions. SAJA employs a balanced loss function combining negative Q-value with an action distance term (Maximum Action Difference—MAD), which proves that maximizing the multi-agent action difference is a principled objective for degrading team value. [41]

Empirical results demonstrate SAJA’s effectiveness: reward drops reach up to 9.90% on 3-agent FACMAC, and SAJA bypasses defenses including M3DDPG, PAAD, and ATLA. The synergy ratio of joint attack impact to sum of individual attack impacts exceeds one confirming that joint state-action perturbations are more potent. [41]

Policy Adversarial Actor-Director (PA-AD): Rather than directly learning state perturbations in high-dimensional spaces, PA-AD proposes a Policy Adversary MDP (PAMDP) where an optimal policy induces the optimal state adversary. The attack employs collaborative "director" and "actor" components: the director learns which policy perturbation directions are most effective, while the actor crafts state perturbations to realize those directions. PA-AD achieves state-of-the-art performance on Atari and MuJoCo tasks, and combined with adversarial training, produces robust policies that significantly outperform baseline defenses.[43]

2.4.2 Action-Space Attacks

Action-space attacks directly manipulate an agent’s output actions, either through direct perturbation of action values or by injecting adversarial agents that execute perturbed actions.

Direct Action Manipulation: SAJA’s action-only variant applies gradient ascent on the critic network conditioned on the perturbed state to craft final adversarial actions. These attacks are particularly effective when attackers possess partial control over one or more agent’s actuators. [41]

Malicious Agent Injection: Rather than perturbing a single agent’s actions, attackers can inject fully adversarial agents into the multi-agent system. These agents learn to disrupt the victim agent’s policies through direct behavioral interference. The Wolfpack attack exemplifies this: by targeting a single agent and subsequently attacking its supporting

agents, adversaries exploit the inherent interdependencies in cooperative systems. The Wolfpack approach reduces rewards by up to 51.8% and severely impacts SMAC scenarios, proving far more effective than random or single-agent attacks. [44]

Adversarial Minority Influence (AMI): AMI leverages social psychology principles where a minority (adversary) unilaterally sways the majority (victims) toward worst-case failure. AMI employs two innovations: (1) a unilateral influence filter adapted from mutual information that maximizes adversary-to-victim influence while excluding reverse influence, and (2) a targeted adversarial oracle that learns worst-case target actions. AMI successfully attacks real robot swarms and simulation environments including StarCraft II and multi-agent MuJoCo. [39]

2.4.3 Communication-Space Attacks

Multi-agent systems with explicit communication channels face a distinct attack surface: adversarial manipulation of inter-agent messages.

Message Hijacking: The paper "Hijacking Robot Teams Through Adversarial Communication" by Wu et al.[45] demonstrates that an adversarial policy can intercept and modify messages between agents. The authors report several-fold performance degradation (e.g., more than four-fold in some simulated predator-prey tasks and roughly a factor of two in Robotarium experiments), illustrating that adversarial communication can catastrophically disrupt team performance. The attack bypasses agents' native policies because the manipulated messages are indistinguishable from benign communication within the agents' learned policies.

Communication-Robust Learning: Advanced attacks model adversarial communication as a cooperative problem, learning adaptive attack strategies that persistently disrupt coordination. The work on differentially private communication reveals that privacy-preserving mechanisms, while protecting against eavesdropping, do not inherently defend against adversarial message perturbations.[46, 47]

2.4.4 Poisoning and Backdoor Attacks

Training-time attacks embed long-term vulnerabilities into learned policies. [13]

Model Poisoning via MARL Coordination: In federated learning with multiple poisoning attackers, attackers coordinate using MADDPG itself to craft effective global model updates that degrade overall system performance. The attack assumes attackers can approximate the benign data distribution, then formulate the poisoning problem as an MDP where attackers jointly learn attack policies through MADDPG, outperforming baseline poisoning methods. [48]

Backdoor Attacks (TrojDRL, BACKDOORL, BadRL, MARLNet): These frameworks embed trigger conditions during training. Once triggered, agents execute attacker-specified actions while appearing normal otherwise. MARLNet specifically targets multi-agent scenarios using imperceptible triggers and reward manipulation. Advanced backdoor

frameworks achieve near-perfect attack success rates (ASR approximating 99%) with minimal utility loss on benign tasks. Backdoor Vaccination, where defenders inject controlled triggers—offers a proactive defense by leveraging the backdoor concept itself. [13, 49–51]

2.4.5 Transferability and Black-Box Attacks

Domain Randomization for Transfer: Adversarial attacks trained on diverse source environments transfer to unseen target policies. Domain randomization across action spaces, reward functions, transition dynamics, policy algorithms, and network structures enables black-box or gray-box attacks without knowledge of target specifics. Interestingly, attacks trained on randomized domains sometimes outperform white-box attacks on the target, suggesting that robust attacks develop from exposure to environmental diversity. [52]

Adversarial Minority Influence (Black-Box): AMI operates without model access, learning attack policies through repeated interaction. The unilateral influence filter enables the attacker to focus on influencing victims while disregarding reverse effects, making the attack practical in black-box settings.[39]

2.4.6 FGSM

Fast Gradient Sign Method (FGSM) [10] is an efficient single-step gradient based technique for generating adversarial examples, it leverages the adaptive nature of neural networks to create small perturbations that maximize the loss increase. It is defined as follows:

$$\eta = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) \quad (2.3)$$

where:

- θ denotes the model parameters
- x represents the input
- y is the target label
- $J(\theta, x, y)$ is the cost function used to train the model

The resulting adversarial example is obtained as:

$$\tilde{x} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) \quad (2.4)$$

where ϵ is a small scalar controlling perturbation magnitude, chosen to remain imperceptible to human observers while remaining sufficient to fool the classifier.

This means that FGSM not only misleads models but makes them confident in the mistakes that they make, successfully "convincing" them that their misclassifications are correct with a significant success rate[15]. Goodfellow et al. [10] propose that FGSM's

efficacy arises from the inherent linearity of neural networks in high-dimensional spaces, fundamentally contradicting the prevailing hypothesis that adversarial vulnerabilities stem from deep network non-linearity.

FGSM efficiency and simplicity have made it a widely used baseline for development of a wide range of sophisticated adversarial attack methods. [53]

Rozsa et al. (2016) [54] extends FGSM by introducing the hot/cold (HC) approach, the hot/cold method operates at the penultimate layer, constructing a feature vector that simultaneously boosts value of features of a target "hot" class and reduces value of features of the original "cold" class enabling backpropagation to generate perturbations that push samples toward different target classes.

Kurakin et al. (2017) [55] proposes the Iterative Fast Gradient Sign Method (I-FGSM), the core methodology generates adversarial examples dynamically during training: at each iteration, adversarial examples are produced from clean samples using the current network state and mixed into minibatches with clean examples, enabling balanced gradient updates through a weighted loss function.

Xie et al. (2019) [56] proposes Diverse Inputs Iterative Fast Gradient Sign Method (DI²-FGSM), a principled extension of FGSM that addresses the white-box-black-box transferability trade-off inherent in the FGSM family of attacks. Rather than directly maximizing the loss with respect to original inputs, as in I-FGSM [55], DI²-FGSM applies stochastic, differentiable transformations (random resizing and padding) at each iteration with probability p , maximizing loss with respect to these transformed inputs.

2.4.7 PGD

Projected Gradient Descent (PGD) is an iterative adversarial attack method that extends FGSM by applying multiple smaller perturbation steps. Projected Gradient Descent (PGD) is introduced by Madry et al. [11] as a multi-step, adversarial attack that iteratively updates an input within a constrained perturbation set to maximize loss. Concretely, for an allowed perturbation set S (typically an ℓ_∞ -ball around a clean input x), PGD starts from an initial point $x_0 \in x + S$ and repeatedly applies gradient ascent on the loss with respect to the input followed by projection back onto $x + S$ following the negative loss function:

$$x_{t+1} = \Pi_{x+S}(x_t + \alpha \text{sign}(\nabla_x L(\theta, x_t, y))),$$

thereby seeking an adversarial example that attains high loss while respecting the perturbation budget. They define budget types in the following categories:

- L_∞ : This constraint sets a maximum modification budget for every pixel. By this definition FGSM is an L_∞ single-step version of PGD.
- L_2 : This constraint sets a maximum modification budget based on the Euclidean distance between the original and adversarial images.

- L_1 : This constraint sets a maximum modification budget based on a set limit of which the sum of all modifications cannot exceed. (Not tested but implicitly compatible with PGD)

Through extensive experiments with random restarts, the authors demonstrate that PGD reliably finds local maxima whose loss values are tightly concentrated, and thus regard PGD as a "universal first-order adversary", meaning that robustness to PGD is indicative of robustness to any attack that only uses first-order (gradient) information. [11]

A survey by Akhtar et al. (2021) [53] highlights a phenomenon called "label leaking" observed in FGSM adversarial training where the model actually overfits for adversarial images resulting in higher accuracy on adversarial examples than on clean images. PGD's method avoids this issue by using random starts that increase variety of modifications. [11]

2.4.8 Overview of adversarial attacks

This subsection is for a summary and ease of reference of the attacks mentioned previously. This is by no means an exhaustive list of adversarial attacks, but should give a good overview of the state of the art.

Table 2.1: Taxonomy of Adversarial Attacks on MADDPG

Attack Domain	Attack Name	Mechanism	Efficacy
Observation Space	FGSM/PGD	Single/iterative gradient-based state perturbations	Limited effectiveness; imperfect critic gradients
	SAJA	Two-phase gradient ascent on state then action with Maximum Action Difference (MAD) loss	9.90% reward drop (3a-FACMAC); synergy ratio > 1 ; bypasses M3DDPG, PAAD, ATLA
	PA-AD	Policy Adversary MDP with director-actor collaboration	SOTA on Atari/MuJoCo; outperforms FGSM/PGD
Action Space	Direct Action Manipulation	Gradient ascent on critic for action perturbations	Effective with partial actuator control; synergistic with state attacks
	Wolfpack Attack	Coordinated multi-stage targeting; exploit inter-agent dependencies	Up to 51.8% reward reduction; severe SMAC impact
	AMI (Adversarial Minority Influence)	Unilateral influence filter + targeted adversarial oracle	Attacks robot swarms, StarCraft II, MuJoCo; black-box capable
Communication Space	Message Hijacking	Intercept and modify inter-agent messages	465% reduction (sim); 201% (physical robots on Robotarium)
	Adaptive Communication Attacks	Learn persistent coordination disruption strategies	Bypasses privacy mechanisms; adaptive to learned patterns

Table 2.2: Taxonomy of Adversarial Attacks on MADDPG part 2

Attack Do-main	Attack Name	Mechanism	Efficacy
Poisoning & Backdoor	Model Poisoning (Federated)	Attackers coordinate via MADDPG to craft poisoned global updates	Outperforms baseline poisoning; coordinated attack via MDP
	Backdoor Attacks	Embed imperceptible triggers during training; execute on trigger	ASR $\approx 99\%$; minimal utility loss on benign tasks
	PNAct (Safe RL Backdoor)	Positive/negative action samples toggling safe/unsafe behavior	Stealthy; increases safety violations while maintaining reward
Transfer & Black-Box	Domain Randomization Transfer	Train on diverse environments; generalize to unseen targets	Transfers across policies, networks, action spaces; sometimes outperforms white-box
	AMI (Black-Box)	Learn attack policies via repeated interaction without model access	No model knowledge required; effective on real swarms

2.5 Graph Neural Networks

Graph Neural Networks (GNNs) are a family of deep learning models designed to operate directly on graph-structured data by iteratively propagating and aggregating information along edges, so that each node (or graph) learns a representation that encodes both its own attributes and the topology of its neighborhood. Their core ideas are based on these principles: Local connectivity, weight sharing, and hierarchical feature extraction. Modern surveys typically define GNNs as neural architectures that take a graph as input and learn task-specific node, edge, or graph-level outputs via repeated “message passing” or neighborhood aggregation, distinguishing them from shallow network embedding methods that merely learn fixed vector lookups without parameter sharing across nodes. [57–60]

Across the four surveys, the most common taxonomy divides GNNs into recurrent GNNs, convolutional GNNs, graph autoencoders, and spatio-temporal GNNs (with attention-based and message-passing variants largely falling under the convolutional category). Recurrent GNNs (often called “GNNs” in the early literature) define node states via recursive equations that iteratively update each node’s hidden representation using its own previous state and the states of its neighbors until a fixed point is reached, followed by a task-specific readout function. Their main advantage is conceptual generality: they naturally handle arbitrary graph structures (directed/undirected, cyclic/acyclic) and can, in principle, approximate complex relational dependencies by iterating message

passing to convergence. However, they are computationally expensive, require contraction conditions for guaranteed convergence, and are difficult to scale to large graphs because many recurrent steps and storage of intermediate states are needed; these limitations largely motivated the shift to feedforward, layer-stacked convolutional GNNs. [57–60]

Convolutional GNNs (CGNNs) extend convolution from regular grids to graphs by defining each layer as a localized aggregation over a node’s neighborhood. [61, 62] In the spectral view, this operation is grounded in graph signal processing, with explicit notions of frequency, smoothness, and graph filters. Polynomial filter approximations lead to spatially localized convolutions and practical implementations, but exact spectral methods are typically expensive and often restricted to transductive settings on a fixed graph. [57–60] As a result, classical spectral CGNNs offer strong theoretical foundations but are less natural to transfer across graphs with different topologies.

Spatial CGNNs (such as GraphSAGE and message-passing neural networks) instead define convolutions directly in the node domain by aggregating neighbor features with permutation-invariant functions (e.g., mean, sum, max, attention), often combined with neighborhood sampling to control complexity.[63, 64] Because the aggregation functions are shared across nodes and graphs, these models scale better to large graphs and naturally support inductive generalization to unseen nodes or entirely new graphs.[57–60] At the same time, deep stacks of spatial layers can suffer from over-smoothing, large-graph performance can depend heavily on the sampling strategy, and the theoretical analysis is generally less mature than in strictly spectral formulations.

Graph autoencoders (GAEs) form another major family. An encoder GNN maps nodes or graphs into a latent space, and a decoder attempts to reconstruct structural information such as edges or the adjacency matrix, often under an unsupervised or self-supervised loss.[65, 66] This view unifies many earlier network embedding methods within a single deep architecture: the learned representations capture topology and attributes and can then be reused for downstream tasks such as clustering, link prediction, or node classification.[57–60] GAEs are particularly useful when labels are scarce, and variational extensions allow modeling distributions over graphs. Their main drawbacks are the cost of decoders that operate on dense adjacency matrices, the risk that pure reconstruction objectives over-prioritize structural fidelity over task-relevant semantics, and sensitivity to design choices such as negative sampling and the balance between observed and unobserved edges.

Spatio temporal GNNs (STGNNs) extend these architectures to graphs with time varying signals or topology by combining graph convolutions with temporal models such as recurrent networks, temporal convolutions, or attention mechanisms. [67, 68] They are now a standard choice for applications like traffic forecasting and human action recognition, where the graph structure is relatively stable but node features evolve over time.[57, 59] The main benefit is that STGNNs can model how information propagates through both space and time—for example, how congestion spreads across a road network—in an end-to-end trainable way. The cost is increased architectural and computational complexity, a greater

need for labeled time-series data, and more challenging regularization and interpretability due to the combined depth in the spatial and temporal dimensions.

Overall, recent surveys converge on the view that GNNs are message-passing architectures on graphs, with recurrent, convolutional, autoencoding, and spatio-temporal variants offering different trade-offs between expressivity, scalability, transferability, and ease of training. [57–60]

2.5.1 GNN applications in network routing

Graph Neural Networks (GNNs) have emerged as a promising tool for learning routing and traffic engineering (TE) policies directly from network structure and traffic, with an explicit goal of generalizing across topologies and traffic conditions. Existing work can be organized into three main application axes: (i) GNNs as performance models for routing, (ii) GNN+RL controllers for online routing/TE, and (iii) specialized, generalizable GNN architectures for large-scale TE and domain-specific routing. [69]

Early work focuses on using GNNs as surrogate models of network behavior, predicting performance metrics for given topologies, traffic variations, and routing configurations. Rusek et al. (2019) [70] show that message-passing GNNs can accurately estimate per-link delay and loss, enabling fast evaluation of many candidate routing schemes without full simulation. Subsequent studies, summarized in Jiang et al. (2021) [69], extend this idea by feeding GNN predictions into external optimizers or discovery based search procedures, transforming TE into a learn, predict, optimize loop. This line of work primarily treats routing as a supervised regression problem, with GNNs learning a differentiable mapping from graph + routing configuration to performance.

A second stream couples GNN encoders with deep reinforcement learning (DRL) to directly learn routing or TE policies. G-Routing proposes a flexible online routing framework where a GNN encodes the current topology, traffic matrix, and routing state, predicting future path-level metrics that a DRL agent then uses to choose routes. [71] Experiments show that G-Routing converges faster and yields lower jitter and more stable performance than shortest-path and traditional TE baselines under traffic dynamics and failures. [71] Similarly, GROM (Generalized Routing Optimization Method) uses a GNN+DRL design to learn traffic-splitting policies; the authors encode split ratios as link features and discretize the continuous action space into classification, improving training stability and generalization to unseen topologies. [72]

More recent works [73] explicitly target generalization across networks and demand patterns by designing GNN architectures aligned with classical TE formulations. A representative approach constructs a TE-GNN whose graph includes paths, source–destination demand pairs, links, and a global objective node, effectively embedding the linear program structure of TE into a heterogeneous graph. Message passing over this graph allows the GNN to learn correlations between paths, links, traffic and the global objective (e.g., maximum link utilization), and experiments indicate that such models can generalize TE

policies to unseen topologies while approximately respecting constraints.[73] Jiang et al. (2024) [74] broader survey situates these models within a larger family of GNN-based control and optimization methods, highlighting that TE and routing are a well studied application for GNNs.

2.6 Taxonomy on Adversarial attacks on GNNs

This subchapter will present contemporary research on adversarial attacks against Graph Neural Networks (GNNs), characterizing them in 6 categories. The classification scheme addresses the unique challenges posed by graph-structured data and the message-passing mechanisms of GNNs that attackers must address in their strategies.

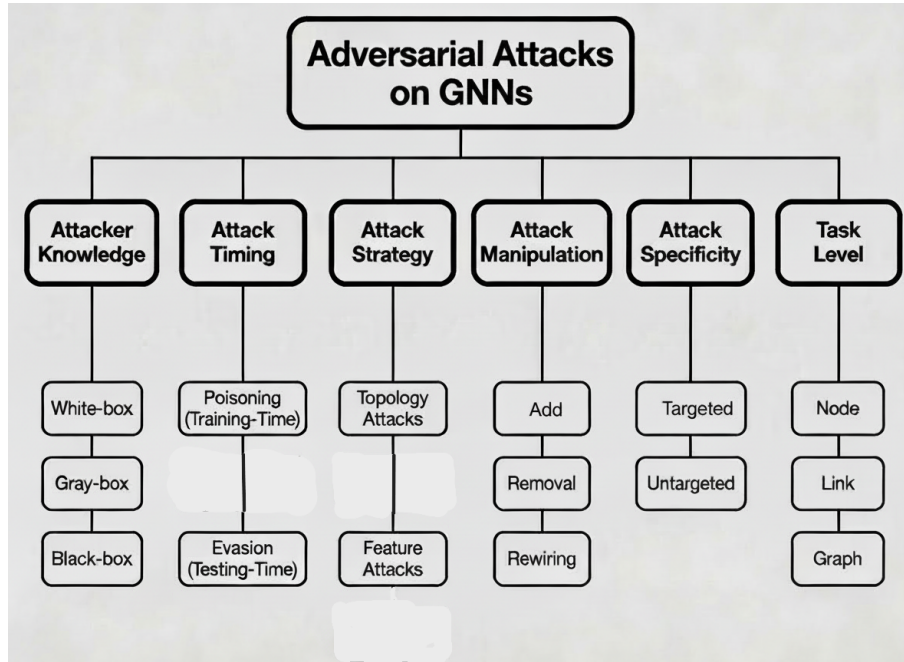


Figure 2.1: Diagram of Adversarial Attack characteristics on Graph Neural Networks

2.6.1 Attacker knowledge

Adversarial attacks on GNNs can be categorized based on the attacker's knowledge:

Table 2.3: Attacker’s Knowledge (Information Access Level)

White-Box	Gray-Box	Black-Box
Complete model access: parameters, architecture, training data, labels	Limited access: training data and labels available; model parameters obscured	Minimal access: only adjacency matrix, feature matrix, and query access to model outputs
Gradient-based optimization feasible; represents worst-case vulnerability assessment	Requires surrogate model training	Query-efficient methods: reinforcement learning, genetic algorithms, or transfer from surrogate models

2.6.2 Attack Timing

Adversarial attacks on GNNs can be categorized by when they are deployed:

- **Training-Time Attacks (Poisoning Attacks):** Adversarial examples are injected into training data to manipulate the model’s learned representations. [75, 76]
- **Test-Time Attacks (Evasion Attacks):** Adversarial examples are introduced during test time to interfere with model predictions. [75, 76]

2.6.3 Attack Strategy (Perturbation Goal)

Adversarial attacks on GNNs can be categorized based on their perturbation goals:

- **Topology Attacks:** Manipulate the graph structure by adding or removing edges to mislead the GNN.
- **Feature Attacks:** Alter node features to deceive the GNN’s predictions.
- **Hybrid Attacks:** Combine topology and feature perturbations to maximize attack effectiveness.

2.6.4 Attack Manipulation

Adversarial attacks on GNNs can be categorized based on the manipulation method whose effectiveness is generally evaluated by stealth rate while maintaining success rates:

- **Addition:** Introduce new nodes or edges to the graph.
- **Removal:** Delete existing nodes or edges from the graph.
- **Rewiring:** A mix of addition and removal.

Rewiring operations tend to preserve graph statistics (node count, edge count, degree distribution) while enabling structural perturbations, making them significantly less detectable. [77]

2.6.5 Attack Specificity

Adversarial attacks on GNNs can be categorized based on their specificity:

- **Targeted Attacks:** Aim to degrade specific model functions or characteristics.
- **Untargeted Attacks:** Seek to degrade overall model performance without targeting specific functions.

2.6.6 Task level

Adversarial attacks on GNNs can be categorized based on the level in which they operate:

- **Node-Level Attacks:** Target individual nodes to mislead node classification or regression tasks. A key vulnerability at this level in GNNs is the assumption of local homophily. [75]
- **Link-Level Attacks:** Focus on specific edges to disrupt link prediction tasks. At this level a common problem is the triadic closure disruption where nodes sometimes create links between them despite having another node which connects them, effectively creating a "triangle" [77]
- **Graph-Level Attacks:** Aim to alter entire graph structures to affect graph classification tasks. An exploitable vulnerability at this level is the graph pooling instability. [77]

2.6.7 Examples and characteristics of adversarial attacks on GNNs

This is by no means a thorough list of adversarial attacks on GNNs, but should give a good overview of the state of the art.

Table 2.4: Summary of Adversarial Attacks on GNNs

Attack Method	Knowledge	Capability	Strategy	Manipulation	Goal	Task
Nettack [78]	White-box	Evasion	Topology	Rewiring	Targeted	Node
Metattack [79]	White-box	Poisoning	Topology	Rewiring	Untargeted	Node
FGA [80]	White-box	Evasion	Topology	Rewiring	Targeted	Node
RL-S2V [81]	Black-box	Evasion	Topology	Rewiring (edge flips)	Targeted	Node/Link
PGD-Attack [82]	White-box	Evasion	Topology	Rewiring	Both	Node
NIPA [83]	Gray-box	Poisoning	Hybrid	Inject nodes	Targeted	Node
ReWatt [84]	Black-box	Evasion	Topology	Rewiring	Untargeted	Graph
IG-FGSM [85]	White-box	Evasion	Hybrid	Rewiring(Feature+Edge)	Both	Node

[75, 76]

In the literature on graph adversarial learning, Nettack and Metattack are typically implemented with full access to model parameters and gradients, which designates them as white-box attacks under standard threat-model definitions [78, 79]. However, some more recent analyses describe it as gray-box when emphasizing that the optimized perturbations are later transferred to a separate victim models. [75, 76]

2.6.8 Transferability-Focused Attacks

Recent work demonstrates that adversarial examples transfer across GNN architectures, datasets, and even task levels. GOTack leverages graph orbit learning to generate universal perturbations effective against multiple GNN variants. Migratability studies show 70-85% attack success rate when transferring between GCN, GAT, and GraphSAGE. [86, 87]

METHODOLOGY

Companion studies and have shown that a MADDPG based network is robust against FGSM style attacks, individual agents attacking all links in a network during every step is ineffective despite having white-box access to reward and critic structures. We will investigate whether a PGD style attack might be more effective, by implementing coordinated multi-agent attacks instead individual agent efforts, and by implementing strategically timed attacks during naturally occurring link failures instead of attempting to brute force every step.

3.1 Starting Point and Scope of This Work

This work is the third stage of a line of research, and it is useful to state at the outset which components are inherited and which are contributed here, since the research question is comparative and therefore depends on the earlier stages remaining unchanged.

3.1.1 The routing system under test

The victim is a multi-agent routing controller developed in earlier work within the same research line [88]. It applies Multi-Agent Deep Deterministic Policy Gradient (MADDPG) to adaptive packet routing under the paradigm of centralised training with decentralised execution (CTDE) [37]: each agent i holds an actor π_{θ_i} that conditions only on its own local observation, while its critic Q_{ϕ_i} may, during training, observe the joint state and the actions of every other agent. Conditioning the critic on the joint action is what stabilises learning, since from any single agent’s perspective the environment would otherwise appear non-stationary as the other agents’ policies evolve [37]. After training the critics are discarded and each agent acts on local information alone. In this routing formulation every forwarding node is an agent whose observation summarises local link utilisation, queue occupancy and the destinations awaiting service, and whose action selects, for each destination, one of K pre-computed candidate paths; the trained system is thus a distributed controller in which no node holds a global view, yet the paths chosen collectively determine end-to-end delivery. That prior work also investigated critic design

and graph-based state encoding, which is the origin of the variant family evaluated in Section 3.2.

This thesis does not modify the routing controller. The trained agents are treated as a fixed system under test: their parameters are frozen, and the adversary interacts with them only through forward evaluations and gradients taken with respect to their inputs.

3.1.2 The FGSM baseline

The second stage established the adversarial setting. A single-step attack based on the Fast Gradient Sign Method (FGSM) [10] was developed against these agents in a preceding dissertation in the same line of work [TODO-colleague-fgsm]. FGSM perturbs an input one step along the sign of the gradient of a loss with respect to that input, $\eta = \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$, under an ℓ_∞ budget $\|\eta\|_\infty \leq \varepsilon$. Applied to the routing controller, the victim is the trained actor, the input is an agent’s local observation, and the objective rewards actions that degrade routing performance rather than ones that induce a misclassification. The threat model it fixed is *observation-space* rather than *actuation-space*: the adversary corrupts what the agents perceive, modelling a compromised telemetry or monitoring plane, while the network itself continues to evolve according to its true state.

That stage contributed not only an attack but an evaluation protocol — the perturbation budget sweep, the frozen-victim assumption, and the metrics by which an attack is judged — and it returned a negative result. The FGSM perturbation alters a substantial fraction of the routing decisions taken by the victim, yet recovers only a small part of the degradation achievable in principle, because the redundancy of the K -path topology absorbs the redirected traffic and leaves end-to-end delivery largely intact.

3.1.3 Contribution of this thesis

That negative result admits two readings: either the controller is genuinely robust, or a single-step, per-agent adversary is too weak for its failure to constitute evidence of robustness. Distinguishing between them requires a stronger adversary, and supplying one is the contribution of this work. Three extensions are developed, all within the inherited threat model: replacing the single gradient step with an iterated projected-gradient optimisation [11]; coordinating the perturbation jointly across compromised agents rather than treating each in isolation (Section 3.3.2); and concentrating a limited attack budget on high-leverage moments rather than spending it uniformly (Section 3.3.3).

Because the purpose is comparison rather than a stronger attack in isolation, compatibility with the earlier stage is a design constraint rather than a convenience. The attacks developed here operate on the same frozen agents, within the same ℓ_∞ ball and the same domain constraints, and are scored under the same evaluation protocol as the FGSM baseline. Any difference in measured damage is then attributable to the attack algorithm itself, rather than to a change in the adversary’s capabilities or in the conditions

of measurement; where this constraint forced a specific implementation choice, it is noted at the relevant point in the text.

3.2 Victim Architectures Under Test

The attacks are evaluated against the family of trained variants inherited from the base system [88] rather than against a single model, so that conclusions about the routing task can be separated from conclusions about one particular network design. All variants share the same actor: a multilayer perceptron mapping the local observation to n_d blocks of K path scores, with the executed route given by the 2 max within each block. They differ along three binary axes, described below.

3.2.1 Central (CC) versus local (LC) critic

The two critic domains differ in what the value function is allowed to observe during training:

$$\underbrace{Q_{\phi^i}(\mathbf{s}, a_1, \dots, a_N)}_{\text{CC: } N(d_{\text{obs}} + |\mathcal{A}|)} \quad \text{versus} \quad \underbrace{Q_{\phi^i}(s_i^{\text{local}}, a_i)}_{\text{LC: } d_{\text{obs}} + |\mathcal{A}|}, \quad (3.1)$$

with d_{obs} the per-agent observation dimension and $|\mathcal{A}| = n_d K$ the per-agent action dimension. The central critic is the canonical MADDPG construction [37]: conditioning on the joint action removes the non-stationarity each agent would otherwise face as its peers' policies change, at the cost of an input that grows linearly in the number of agents. The local critic reduces the scheme to independent actor-critic learners; it forfeits that guarantee but is far cheaper and its size is independent of the agent count.

The distinction also determines which attack objectives are available. Because the central critic is an explicit function of *all* agents' actions, it supplies a non-separable objective that couples the compromised agents, used in Equation (3.11). Local-critic variants admit no such objective and are attacked with the congestion objective of Equation (3.12) instead.

3.2.2 Simple versus duelling value head

The simple head passes the concatenated state and action through two rectified hidden layers and emits a scalar. The duelling head replaces the output layer with two parallel streams whose outputs are summed, $Q(s, a) = V(h) + A(h)$.

One caveat should be stated. In the original duelling formulation the advantage stream emits one value *per action*, and subtracting the mean advantage makes the split identifiable. Here the action is an *input* to the critic rather than an output index, so both streams emit scalars and no such constraint is applied: any constant may be shifted between V and A without changing Q . The duelling head is thus an extra parallel projection rather than a value-advantage separation, and differences it produces should be attributed on that basis.

3.2.3 With and without GNN encoding

The GNN variants insert a shared encoder between the environment and the actor. Each agent is a graph node whose features are its full observation, edges come from the trainable subgraph of the physical topology, and two GraphConv layers aggregate neighbour information under a residual connection:

$$\tilde{\mathbf{X}} = \text{ReLU}\left(\text{GraphConv}_2(\text{ReLU}(\text{GraphConv}_1(\mathbf{X}, E)), E) + \mathbf{X}\right). \quad (3.2)$$

The encoder is dimension-preserving, so it is a drop-in pre-processor and the actor is unchanged between variants, and the residual connection makes it a near-identity transform at initialisation. Optional relay nodes representing transit switches carry zero features and act purely as message-passing intermediaries, propagating observations two hops through the topology.

The consequence for the attacks is structural rather than one of degree: without the encoder an agent’s policy reads its own observation alone, whereas with it a perturbation applied to one agent reaches its neighbours’ encoded states before their actors. The single-agent differentiable path assumed by the critic-grounded attack therefore does not hold for these variants, which must be attacked with the joint formulation of Section 3.3.2 or excluded from that comparison.

3.2.4 Variant matrix

Table 3.1: MADDPG victim variants under test, named <critic domain>-<value head>[-GNN].

Variant	Critic	Value head	GNN
CC-Simple	Central	Simple	—
CC-Duelling	Central	Duelling	—
CC-Simple-GNN	Central	Simple	Yes
CC-Duelling-GNN	Central	Duelling	Yes
LC-Simple	Local	Simple	—
LC-Duelling	Local	Duelling	—
LC-Duelling-GNN	Local	Duelling	Yes

The matrix is not the full $2 \times 2 \times 2$ factorial: LC-Simple-GNN is omitted, as the local-critic value-head comparison is already covered by LC-Simple/LC-Duelling and the GNN comparison by LC-Duelling/LC-Duelling-GNN. Where a single variant is reported, CC-Simple serves as the reference configuration, being the canonical MADDPG formulation with no additional architectural machinery.

3.3 PGD-Based Adversarial Attacks on MADDPG Routing Agents

Formally, given a clean input x , label y , model parameters θ , loss function $L(\theta, x, y)$, a perturbation budget ε , a step size α and a norm constraint set

$$\mathcal{S}(x, \varepsilon) = \{x' \mid \|x' - x\|_p \leq \varepsilon\}, \quad (3.3)$$

PGD generates an adversarial example x^{adv} by iterating

$$x^{(0)} = x + \delta_0, \quad \delta_0 \sim \text{Unif}(\mathcal{S}(0, \varepsilon)), \quad (3.4)$$

$$x^{(t+1)} = \Pi_{\mathcal{S}(x, \varepsilon)} \left(x^{(t)} + \alpha \cdot \text{sign}(\nabla_x L(\theta, x^{(t)}, y)) \right), \quad (3.5)$$

where $\Pi_{\mathcal{S}(x, \varepsilon)}(\cdot)$ denotes the projection operator onto the set $\mathcal{S}(x, \varepsilon)$ and the sign operator corresponds to ℓ_∞ -constrained PGD [11]. After T steps, $x^{\text{adv}} = x^{(T)}$ is used as the adversarial example.

We adapt PGD to the MADDPG routing environment by implementing two key features on top of the FGSM baseline.

- **(A) Coordinated Multi-Agent Perturbation:** By replacing the independent per-agent loop with a joint adversary we can direct several compromised agents to a single shared surviving link.
- **(B) Strategically-timed / critical-state attack:** By providing a L0 budget (limited attack budget) and monitoring of the network with a critical-state score, we can trigger the attack at critical moments when network failsafe redundancies are thinner.

3.3.1 Threat model and admissible perturbation set

Both extensions share a single threat model, which is deliberately identical to the one used for the FGSM baseline so that any observed difference in damage is attributable to the attack itself rather than to a change in the attacker’s capabilities.

Let $C \subseteq \{1, \dots, M\}$ denote the set of compromised agents and $N = |C|$ its cardinality. The victim policies $\{\pi_{\theta_i}\}$ and critics $\{Q_{\phi^i}\}$ are *frozen*: their parameters are fixed throughout, and the adversary interacts with them only through forward evaluations and gradients taken with respect to the *input* observations. No victim parameter is ever updated, so the attack measures the robustness of a fixed trained policy rather than the outcome of an adversarial training game.

The adversary perturbs observations only; the underlying network state evolves from the true, unperturbed dynamics. For agent i at time t the admissible set is

$$\mathcal{S}(s_{i,t}^{\text{local}}, \varepsilon) = \left\{ \hat{s} \mid \|\hat{s} - s_{i,t}^{\text{local}}\|_\infty \leq \varepsilon, \quad \hat{s} \in \mathcal{D} \right\}, \quad (3.6)$$

where $\mathcal{D}$ is the domain constraint that re-projects the normalised bandwidth features onto $[0, 1]$. The corresponding projection operator $\Pi_{\mathcal{S}}$ therefore composes an ℓ_∞ clip about

the clean observation with the same domain clip used by the FGSM baseline. Because the measured damage depends on the size of the admissible set, the *same* operator Π_S is applied to every attack and to the random control of Section 3.3.4; using a stricter domain clip for one attack than another would confound the perturbation direction with the size of the set it is drawn from.

3.3.2 (A) Coordinated multi-agent perturbation

Separability of the per-agent attack

The FGSM baseline, and any attack that iterates over agents independently, solves N decoupled problems: for each $i \in C$,

$$\eta_{i,t}^* = 2 \max_{\eta: s_{i,t}^{\text{local}} + \eta \in S} \mathcal{L}_i(s_{i,t}^{\text{local}} + \eta), \quad (3.7)$$

where $\mathcal{L}_i$ depends only on agent i 's own observation and policy. The joint perturbation obtained this way is the concatenation of N independent solutions, and the implied joint objective is *separable*,

$$\mathcal{J}_{\text{sep}}(\hat{\mathbf{s}}_t) = \sum_{i \in C} \mathcal{L}_i(\hat{s}_{i,t}), \quad (3.8)$$

so no term couples two agents. This is a structural limitation rather than a matter of attack strength: a separable objective cannot express a preference for outcomes that depend on *which* agents are steered onto the *same* resource, because the value of steering agent i onto a link is independent of whether agent j was steered onto it as well. In a topology whose robustness comes from K -path redundancy, the damaging configurations are precisely those in which several redirected flows converge on one surviving link, and these lie outside the reach of Equation (3.8).

Joint formulation

We therefore optimise over the joint perturbation directly. Writing $\mathbf{s}_t = (s_{1,t}^{\text{local}}, \dots, s_{N,t}^{\text{local}})$ for the stacked observations of the compromised agents, the coordinated attack solves

$$\hat{\mathbf{s}}_t^* = 2 \max_{\hat{\mathbf{s}} \in \mathcal{S}(\mathbf{s}_t, \varepsilon)^N} \mathcal{J}(\hat{\mathbf{s}}), \quad (3.9)$$

with a *non-separable* global objective $\mathcal{J}$. A single gradient of $\mathcal{J}$ supplies a descent direction for every compromised agent simultaneously, so one optimisation step trades the agents off against one another.

Differentiating through the routing decision

The environment decodes a routing decision by taking, for each destination d , the 2 max over the K candidate paths. The realised route is therefore a piecewise constant function of the observation, and its gradient vanishes almost everywhere, which would leave

Equation (3.9) without usable first-order information. We adopt a straight-through estimator: the forward pass uses the hard per-destination one-hot decision actually executed by the environment, while the backward pass propagates the gradient of the underlying softmax policy,

$$\text{ST}(\pi_{\theta_i}(\hat{s})) = \text{sg} \left[\text{onehot}(\pi_{\theta_i}(\hat{s})) - \pi_{\theta_i}(\hat{s}) \right] + \pi_{\theta_i}(\hat{s}), \quad (3.10)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator and $\text{onehot}(\cdot)$ is applied blockwise, one block of K entries per destination. This choice matters for interpreting the results: the gradient is informative about how to *change a routing decision*, which is not the same quantity as how to *cause congestion*, and the two coincide only when the network cannot absorb the redirected flow.

Attack objectives

Two instantiations of $\mathcal{J}$ are considered. The first is grounded in the victim’s own centralised critics, evaluated on the joint action induced by the perturbed observations,

$$\mathcal{J}_{\text{critic}}(\hat{\mathbf{s}}) = -\frac{1}{N} \sum_{i \in \mathcal{C}} Q_{\phi^i} \left(x_t, (\tilde{a}_1, \dots, \tilde{a}_N) \right), \quad \tilde{a}_j = \text{ST}(\pi_{\theta_j}(\hat{s}_j)), \quad (3.11)$$

where x_t is the true centralised state, held fixed during the attack. Because the centralised critic of the CTDE formulation (Section 3.2.1) takes the actions of *all* agents as input [37], it is by construction a non-separable function of the joint perturbation, and ascending $-Q$ steers the joint action towards the region the victim’s own value model rates as worst.

The second objective targets the shared-bottleneck mechanism explicitly. Let $u_{d,k}^{(i)}$ denote the true bottleneck utilisation of the k -th candidate path to destination d for agent i , read from the *clean* observation, and let $[\tilde{a}_i]_{d,k}$ be the corresponding entry of the straight-through action. Then

$$\mathcal{J}_{\text{cong}}(\hat{\mathbf{s}}) = \sum_{i \in \mathcal{C}} \sum_d \sum_k [\tilde{a}_i]_{d,k} u_{d,k}^{(i)}. \quad (3.12)$$

Summing over agents rewards configurations in which several agents are steered onto the same congested path, which is the mechanism under test. The utilisation coefficients are taken from the clean observation rather than the perturbed one; were they read from $\hat{\mathbf{s}}$, the attacker would be optimising against its own forged view of the network instead of its true state.

Joint projected gradient ascent

Equation (3.9) is solved by projected gradient ascent following the standard PGD construction [11], applied to the stacked perturbation. Starting from a uniformly random point inside the ball,

$$\hat{\mathbf{s}}^{(0)} = \Pi_{\mathcal{S}}(\mathbf{s}_t + \delta_0), \quad \delta_0 \sim \text{Unif}(\mathcal{S}(0, \varepsilon)^N), \quad (3.13)$$

$$\hat{\mathbf{s}}^{(\tau+1)} = \Pi_{\mathcal{S}} \left(\hat{\mathbf{s}}^{(\tau)} + \alpha \cdot \text{sign} \left(\nabla_{\hat{\mathbf{s}}} \mathcal{J}(\hat{\mathbf{s}}^{(\tau)}) \right) \right). \quad (3.14)$$

The random initialisation is what distinguishes PGD from the iterative FGSM variant, which starts from the clean point [55]; the procedure is repeated from R independent random starts and the iterate achieving the highest objective is retained, since the local maxima of the inner problem are distinct but concentrated in value [11]. The step size defaults to $\alpha = 2.5 \varepsilon / T$ for T iterations, which guarantees the boundary of the ball is reachable from any interior starting point [11]. The full procedure is given in Algorithm 1.

Algorithm 1: Coordinated multi-agent PGD (one time step)

Input: clean observations $\mathbf{s}_t$; centralised state x_t ; budget ε ; steps T ; step size α ;
 restarts R ; objective $\mathcal{J}$

Output: adversarial observations $\hat{\mathbf{s}}_{i,t}$ for all $i \in C$

$\mathcal{J}^{\text{best}} \leftarrow -\infty$

$U \leftarrow$ path utilisations read from the clean $\mathbf{s}_t$

for $r \leftarrow 1$ **to** R **do**

$\hat{\mathbf{s}} \leftarrow \Pi_S(\mathbf{s}_t + \delta_0), \quad \delta_0 \sim \text{Unif}(\mathcal{S}(0, \varepsilon)^N) \quad // \text{ random start}$

for $\tau \leftarrow 1$ **to** T **do**

$\tilde{a}_j \leftarrow \text{ST}(\pi_{\theta_j}(\hat{\mathbf{s}}_j))$ for all $j \in C \quad // \text{ one shared graph}$

$J \leftarrow \mathcal{J}(\tilde{a}_{1:N}, x_t, U)$

$g \leftarrow \nabla_{\hat{\mathbf{s}}} J \quad // \text{ one backward} \Rightarrow \text{gradient for every agent}$

$\hat{\mathbf{s}} \leftarrow \Pi_S(\hat{\mathbf{s}} + \alpha \text{sign}(g))$

end

if $\mathcal{J}(\hat{\mathbf{s}}) > \mathcal{J}^{\text{best}}$ **then**

$\mathcal{J}^{\text{best}} \leftarrow \mathcal{J}(\hat{\mathbf{s}}); \quad \hat{\mathbf{s}}^* \leftarrow \hat{\mathbf{s}}$

end

end

return rows of $\hat{\mathbf{s}}^*$

The attack is solved once per time step for the whole compromised group; each agent subsequently receives its own block of the joint solution, so the N agents act on a single coordinated decision rather than on N independent ones.

3.3.3 (B) Strategically-timed attacks under an L_0 budget

The attacks considered so far act at every time step. A more realistic adversary is constrained in *how often* it can act, which converts the attack into a resource-allocation problem over the trajectory. We impose an L_0 budget: for an episode of T_{ep} steps and a budget fraction $b \in (0, 1]$,

$$\frac{1}{T_{\text{ep}}} \sum_{t=1}^{T_{\text{ep}}} \mathbb{I}[\text{attack at } t] \leq b. \quad (3.15)$$

Spending this budget uniformly at random would waste it on steps at which the network has ample spare capacity. We instead spend it at moments of high leverage,

identified by a *critical-state score*. We use the maximum link utilisation across the topology,

$$c_t = \max_{(u,v) \in E} \rho_t(u, v), \quad (3.16)$$

where $\rho_t(u, v)$ is the utilisation of link (u, v) at time t . This single quantity responds to both regimes of interest: congestion onset raises it directly, and the loss of a link raises it indirectly by concentrating traffic onto the surviving paths, so the score is elevated exactly when redundancy is thinnest.

Because the distribution of c_t is not known in advance and drifts within an episode, the trigger threshold is calibrated online. Let W_t be a sliding window of the most recent scores. The adversary acts at time t if

$$c_t \geq \theta_t, \quad \theta_t = Q_{1-b}(\{c_{t'}\}_{t' \in W_t}), \quad (3.17)$$

where Q_{1-b} denotes the empirical $(1 - b)$ -quantile. The threshold is computed from the window *before* c_t is inserted, so that a step never influences its own decision, and the quantile construction makes the realised attack rate track b without requiring the remainder of the episode to be known in advance. A short warm-up period, during which the adversary always acts, seeds the window before the quantile is meaningful. The procedure is summarised in Algorithm 2.

Algorithm 2: Critical-state timing gate

Input: budget b ; window size W ; warm-up length H ; network state at time t
Output: decision $fire \in \{\text{true}, \text{false}\}$
 $c_t \leftarrow \max_{(u,v) \in E} \rho_t(u, v)$ // critical-state score
if $|\mathcal{H}| < H$ **then**
 $fire \leftarrow \text{true}$ // warm-up: seed the window
else
 $\theta_t \leftarrow Q_{1-b}(\mathcal{H})$ // computed before inserting c_t
 $fire \leftarrow (c_t \geq \theta_t)$
end
 insert c_t into $\mathcal{H}$ (ring buffer of size W)
return $fire$

The gate composes with either attack scope: it decides *whether* to act at a given step, while Section 3.3.2 decides *how* the perturbation is distributed across agents once the decision to act has been taken. On skipped steps the agents receive their unmodified observations.

3.3.4 Evaluation protocol

Paired arms

Each configuration is evaluated with three arms: a *clean* arm with no perturbation, a *random* control that draws a uniformly random direction from the same admissible set

of Equation (3.6), and the *attack* under test. All three are run on identical episodes: the traffic sequence and any injected link failures are generated from a common seed, so episode e of one arm and episode e of another differ only in the perturbation applied. The random control draws from a dedicated random stream so that it consumes no entropy from the traffic generator, which would otherwise desynchronise the arms and invalidate the pairing.

Adversarial-specific damage

Under link failures the raw degradation relative to the clean arm is dominated by the failures themselves rather than by the attack. The primary reported quantity is therefore the *adversarial gap*, the paired per-episode difference between the random control and the attack,

$$\Delta^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left(\text{PDR}_e^{\text{rand}} - \text{PDR}_e^{\text{atk}} \right), \quad (3.18)$$

reported with a paired 95% confidence interval, $\Delta^{\text{adv}} \pm t_{0.975, n-1} s_d / \sqrt{n}$, where s_d is the standard deviation of the per-episode differences. Differencing within episode pairs removes the episode-to-episode variance that otherwise dominates the comparison. The raw drop with respect to the clean arm is reported alongside it, but is not used as the measure of attack strength.

Decisions versus outcomes

Two families of metric are recorded and reported separately. *Decision* metrics count how much routing behaviour the attack changed, principally the action flip rate, the fraction of per-agent, per-destination 2 max path choices that differ between the clean and adversarial observations at the same time step,

$$\text{flip} = \frac{1}{|C| n_d T_{\text{ep}}} \sum_t \sum_{i \in C} \sum_d \mathbb{1} \left[2 \max_k \pi_{\theta_i}(\hat{s}_{i,t})_{d,k} \neq 2 \max_k \pi_{\theta_i}(s_{i,t}^{\text{local}})_{d,k} \right]. \quad (3.19)$$

Outcome metrics measure delivered traffic, principally end-to-end PDR. The distinction is essential in a topology with K -path redundancy, where a high flip rate accompanied by an unchanged PDR indicates that the network absorbed the redirected flows rather than that the attack succeeded. Reporting a flip rate as though it were a measure of damage would conflate the two.

STATISTICS AND RESULTS

BIBLIOGRAPHY

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29) (cit. on p. iii).
- [2] J. M. Lourenço. *The aidisclose package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: <https://ctan.org/pkg/aidisclose> (cit. on p. iii).
- [3] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. “GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing”. In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).
- [4] C. Tian, H. Cao, J. Xie, S. Garg, J. Rodrigues, and M. S. Hossain. “MADDPG-empowered slice reconfiguration approach for 5G multi-tier system”. In: *Journal of Network and Computer Applications* 219 (2023-09), p. 103725. DOI: 10.1016/j.jnca.2023.103725 (cit. on pp. 1, 7).
- [5] H. Wang, Y. Liu, W. Li, and Z. Yang. “Multi-agent deep reinforcement learning-based fine-grained traffic scheduling in data center networks”. In: *Future Internet* 16.4 (2024), p. 119 (cit. on p. 1).
- [6] W. Huang, H. Liu, Y. Li, and L. Ma. “ERA-MADDPG: An Elastic Routing Algorithm Based on Multi-Agent Deep Deterministic Policy Gradient in SDN”. In: *Future Internet* 17.7 (2025), p. 291 (cit. on pp. 1, 7).
- [7] S. S. Bhavanasi, L. Pappone, and F. Esposito. “Routing with Graph Convolutional Networks and Multi-Agent Deep Reinforcement Learning”. In: *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. 2022, pp. 72–77. DOI: 10.1109/NFV-SDN56302.2022.9974607 (cit. on p. 1).
- [8] W. Zhang, C. Liu, Y. Pi, Y. Zhang, H. Huang, B. Rao, Y. Ding, S. Yang, and J. Jiang. *DRAMA: A Dynamic Packet Routing Algorithm using Multi-Agent Reinforcement Learning with Emergent Communication*. 2025. arXiv: 2504.04438 [cs.NI] (cit. on pp. 1, 7, 8).
- [9] F. Alanazi and M. Zareei. “Multi-Agent Deep Reinforcement Learning for Dynamic Routing in MANETs Using Graph Neural Networks”. In: *IEEE Access* 13 (2025), pp. 152469–152478. DOI: 10.1109/ACCESS.2025.3601994 (cit. on p. 1).

- [10] I. J. Goodfellow, J. Shlens, and C. Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572 [stat.ML] (cit. on pp. 1, 9, 11, 24).
- [11] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2019. arXiv: 1706.06083 [stat.ML] (cit. on pp. 1, 9, 12, 13, 24, 27, 29, 30).
- [12] N. Carlini and D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. 2017. arXiv: 1608.04644 [cs.CR] (cit. on p. 1).
- [13] J. Dai and H. Sun. *Effective backdoor attack on graph neural networks in link prediction tasks*. 2025. arXiv: 2401.02663 [cs.LG] (cit. on pp. 1, 3, 10, 11).
- [14] C. Wang. *Attacking and Defending Deep Reinforcement Learning Policies*. 2022. arXiv: 2205.07626 [cs.LG] (cit. on p. 1).
- [15] W. Villegas-Ch, A. Jaramillo-Alcázar, and S. Luján-Mora. “Evaluating the robustness of deep learning models against adversarial attacks: An analysis with fgsm, pgd and cw”. In: *Big Data and Cognitive Computing* 8.1 (2024), p. 8 (cit. on pp. 1, 4, 11).
- [16] T. Theodoropoulos, D. Kafetzis, J. Violos, A. Makris, and K. Tserpes. “Multi-agent deep reinforcement learning for weighted multi-path routing”. In: *Proceedings of the 3rd Workshop on Flexible Resource and Application Management on the Edge*. 2023, pp. 7–11 (cit. on p. 3).
- [17] M. M. Alam and S. Moh. “Joint Trajectory Control, Frequency Allocation, and Routing for UAV Swarm Networks: A Multi-Agent Deep Reinforcement Learning Approach”. In: *IEEE Transactions on Mobile Computing* 23.12 (2024), pp. 11989–12005. doi: 10.1109/TMC.2024.3403890 (cit. on p. 3).
- [18] F. Lozano-Cuadra, M. D. Thorsager, I. Leyva-Mayorga, and B. Soret. *An open source Multi-Agent Deep Reinforcement Learning Routing Simulator for satellite networks*. 2024. arXiv: 2407.11047 [cs.LG] (cit. on p. 3).
- [19] P. Liu, X. Bai, and H. Cheng. “DRL-MR: Multipath Routing Based on Multi-Agent Deep Reinforcement Learning for SDN-Based Data Center Networks”. In: *2025 International Joint Conference on Neural Networks (IJCNN)*. 2025, pp. 1–7. doi: 10.1109/IJCNN64981.2025.11227800 (cit. on p. 3).
- [20] R. A. Alliche, R. A. Pardo, and L. Sassatelli. “O-DQR: A Multi-Agent Deep Reinforcement Learning for Multihop Routing in Overlay Networks”. In: *IEEE Transactions on Network and Service Management* 22.1 (2025), pp. 439–455. doi: 10.1109/TNSM.2024.3485196 (cit. on p. 3).
- [21] Z. Jia, S. He, Q. Zhu, W. Wang, Q. Wu, and Z. Han. “Trusted Routing for Blockchain-Empowered UAV Networks via Multi-Agent Deep Reinforcement Learning”. In: *IEEE Transactions on Communications* 73.12 (2025), pp. 14227–14242. doi: 10.1109/TCOMM.2025.3597655 (cit. on p. 3).

-
- [22] Z. Zhou, G. Liu, W. Guo, and M. Zhou. "Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. doi: 10.1109/TSMC.2024.3454118 (cit. on pp. 3, 6).
 - [23] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199 [cs.CV] (cit. on p. 3).
 - [24] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun. *Tactics of Adversarial Attack on Deep Reinforcement Learning Agents*. 2019. arXiv: 1703.06748 [cs.LG] (cit. on p. 3).
 - [25] S. Yamabe, K. Fukuchi, and J. Sakuma. *Robust Deep Reinforcement Learning against Adversarial Behavior Manipulation*. 2025. arXiv: 2406.03862 [cs.LG] (cit. on p. 3).
 - [26] A. Alzubaidi, A. S. Al-Sumaiti, and M. Khonji. "Adversarial Deep Reinforcement Learning Attacks on Multi-Agent Autonomous Cooperative Driving Policies". In: *IET Intelligent Transport Systems* 19.1 (2025), e70066. doi: <https://doi.org/10.1049/itr2.70066> (cit. on p. 3).
 - [27] Z. Zhou, G. Liu, W. Guo, and M. Zhou. "Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. doi: 10.1109/TSMC.2024.3454118 (cit. on p. 3).
 - [28] S. G. Finlayson, J. D. Bowers, J. Ito, J. L. Zittrain, A. L. Beam, and I. S. Kohane. "Adversarial attacks on medical machine learning". In: *Science* 363.6433 (2019), pp. 1287–1289. doi: 10.1126/science.aaw4399 (cit. on p. 3).
 - [29] T. Duan, Z. Zhang, Z. Lin, Y. Gao, L. Xiong, Y. Cui, H. Liang, X. Chen, H. Cui, and D. Huang. "Rethinking Adversarial Attacks in Reinforcement Learning from Policy Distribution Perspective". In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2025, pp. 1–5. doi: 10.1109/ICASSP49660.2025.10890540 (cit. on p. 4).
 - [30] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG] (cit. on pp. 4, 5).
 - [31] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. "Human-level control through deep reinforcement learning". In: *nature* 518.7540 (2015), pp. 529–533 (cit. on p. 4).
 - [32] X. Geng and B. Zhang. "Deep Q-Network-Based Intelligent Routing Protocol for Underwater Acoustic Sensor Network". In: *IEEE Sensors Journal* 23.4 (2023), pp. 3936–3943. doi: 10.1109/JSEN.2023.3234112 (cit. on p. 5).

- [33] Q. Fu, E. Sun, K. Meng, M. Li, and Y. Zhang. “Deep Q-Learning for Routing Schemes in SDN-Based Data Center Networks”. In: *IEEE Access* 8 (2020), pp. 103491–103499. doi: 10.1109/ACCESS.2020.2995511 (cit. on p. 5).
- [34] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. *Dueling Network Architectures for Deep Reinforcement Learning*. 2016. arXiv: 1511.06581 [cs.LG] (cit. on pp. 5, 6).
- [35] M. Zhao. *Breaking the Cold-Start Barrier: Reinforcement Learning with Double and Dueling DQNs*. 2025. arXiv: 2508.21259 [cs.IR] (cit. on p. 5).
- [36] J. Zhang, F. Qian, and J. Yang. “Online routing and spectrum allocation in elastic optical networks based on dueling Deep Q-network”. In: *Computers & Industrial Engineering* 173 (2022), p. 108663. issn: 0360-8352. doi: <https://doi.org/10.1016/j.cie.2022.108663> (cit. on p. 6).
- [37] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. “Multi-agent actor-critic for mixed cooperative-competitive environments”. In: *Advances in neural information processing systems* 30 (2017) (cit. on pp. 6, 23, 25, 29).
- [38] N. H. Pham, L. M. Nguyen, J. Chen, H. T. Lam, S. Das, and L. Weng. “c-MBA: Adversarial attack for cooperative MARL using learned dynamics model”. In: *NeurIPS ML Safety Workshop*. 2022 (cit. on p. 6).
- [39] S. Li, J. Guo, J. Xiu, Y. Zheng, P. Feng, X. Yu, J. Wang, A. Liu, Y. Yang, B. An, et al. “Attacking cooperative multi-agent reinforcement learning by adversarial minority influence”. In: *Neural Networks* (2025), p. 107747 (cit. on pp. 6, 10, 11).
- [40] S. Kuang, J. Zheng, S. Liang, Y. Li, S. Liang, and W. Huang. “RS-MADDPG: Routing Strategy Based on Multi-Agent Deep Deterministic Policy Gradient for Differentiated QoS Services”. In: *Future Internet* 17.9 (2025), p. 393 (cit. on p. 7).
- [41] W. Guo, G. Liu, and Z. Zhou. *SAJA: A State-Action Joint Attack Framework on Multi-Agent Deep Reinforcement Learning*. 2025. arXiv: 2510.13262 [cs.AI] (cit. on p. 9).
- [42] H. Waghela, J. Sen, and S. Rakshit. “Robust Image Classification: Defensive Strategies against FGSM and PGD Adversarial Attacks”. In: *2024 Asian Conference on Intelligent Technologies (ACOIT)*. IEEE, 2024-09, pp. 1–7. doi: 10.1109/acoit62457.2024.10941671 (cit. on p. 9).
- [43] Y. Sun, R. Zheng, Y. Liang, and F. Huang. “Who is the strongest enemy? towards optimal and efficient evasion attacks in deep rl”. In: *arXiv preprint arXiv:2106.05087* (2021) (cit. on p. 9).
- [44] S. Lee, J. Hwang, Y. Jo, and S. Han. *Wolfpack Adversarial Attack for Robust Multi-Agent Reinforcement Learning*. 2025. arXiv: 2502.02844 [cs.LG] (cit. on p. 10).
- [45] Z. Wu, S. C. Ye, B. Han, and M. Gombolay. “Hijacking Robot Teams Through Adversarial Communication”. In: *7th Annual Conference on Robot Learning*. 2023 (cit. on p. 10).

- [46] C. Zhao, Y. Ze, J. Dong, B. Wang, and S. Li. “DPMAC: Differentially private communication for cooperative multi-agent reinforcement learning”. In: *arXiv preprint arXiv:2308.09902* (2023) (cit. on p. 10).
- [47] L. Yuan, F. Chen, Z. Zhang, and Y. Yu. *Communication-Robust Multi-Agent Learning by Adaptable Auxiliary Multi-Agent Adversary Generation*. 2023. arXiv: 2305.05116 [cs.LG] (cit. on p. 10).
- [48] W. Shen, H. Li, and Z. Zheng. “Coordinated attacks against federated learning: A multi-agent reinforcement learning approach”. In: *ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML)*. ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML). 2021 (cit. on p. 10).
- [49] W. Guo, G. Liu, Z. Zhou, and L. Wang. “PNAct: Crafting Backdoor Attacks in Safe Reinforcement Learning”. In: *arXiv preprint arXiv:2507.00485* (2025) (cit. on p. 11).
- [50] J. Guo, A. Li, L. Wang, and C. Liu. “Polycycle: Backdoor detection and mitigation for competitive reinforcement learning”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 4699–4708 (cit. on p. 11).
- [51] S. Chen, Y. Qiu, and J. Zhang. *Backdoor Attacks on Multiagent Collaborative Systems*. 2022. arXiv: 2211.11455 [cs.MA] (cit. on p. 11).
- [52] X. Pan, Y. Cao, X. Wu, E. Zelikman, C. Xiao, Y. Sui, R. Chakraborty, and R. S. Fearing. “Transferable adversarial attacks on deep reinforcement learning with domain randomization”. In: (2020) (cit. on p. 11).
- [53] N. Akhtar, A. Mian, N. Kardan, and M. Shah. “Advances in adversarial attacks and defenses in computer vision: A survey”. In: *IEEE access* 9 (2021), pp. 155161–155196 (cit. on pp. 12, 13).
- [54] A. Rozsa, E. M. Rudd, and T. E. Boult. “Adversarial diversity and hard positive generation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2016, pp. 25–32 (cit. on p. 12).
- [55] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial Machine Learning at Scale*. 2017. arXiv: 1611.01236 [cs.CV] (cit. on pp. 12, 30).
- [56] C. Xie, Z. Zhang, Y. Zhou, S. Bai, J. Wang, Z. Ren, and A. L. Yuille. “Improving Transferability of Adversarial Examples With Input Diversity”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019-06 (cit. on p. 12).
- [57] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun. *Graph Neural Networks: A Review of Methods and Applications*. 2021. arXiv: 1812.08434 [cs.LG] (cit. on pp. 15–17).
- [58] Z. Zhang, P. Cui, and W. Zhu. “Deep Learning on Graphs: A Survey”. In: *IEEE Transactions on Knowledge and Data Engineering* 34.1 (2022), pp. 249–270. doi: 10.1109/TKDE.2020.2981333 (cit. on pp. 15–17).

- [59] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. "A Comprehensive Survey on Graph Neural Networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), pp. 4–24. doi: 10.1109/TNNLS.2020.2978386 (cit. on pp. 15–17).
- [60] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy. "Machine learning on graphs: A model and comprehensive taxonomy". In: *Journal of Machine Learning Research* 23.89 (2022), pp. 1–64 (cit. on pp. 15–17).
- [61] O. Rippel, J. Snoek, and R. P. Adams. "Spectral Representations for Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett. Vol. 28. Curran Associates, Inc., 2015. URL: https://proceedings.neurips.cc/paper_files/paper/2015/file/536a76f94cf7535158f66cfbd4b113b6-Paper.pdf (cit. on p. 16).
- [62] S. Tang, B. Li, and H. Yu. "ChebNet: efficient and stable constructions of deep neural networks with rectified power units via Chebyshev approximation". In: *Communications in Mathematics and Statistics* (2024), pp. 1–27 (cit. on p. 16).
- [63] E. Jiawei, Y. Zhang, S. Yang, H. Wang, X. Xia, and X. Xu. "GraphSAGE++: Weighted multi-scale GNN for graph representation learning". In: *Neural Processing Letters* 56.1 (2024), p. 24 (cit. on p. 16).
- [64] M. W. Raja. "Artificial intelligence-based healthcare data analysis using multi-perceptron neural network (MPNN) based on optimal feature selection". In: *SN Computer Science* 5.8 (2024), p. 1034 (cit. on p. 16).
- [65] Z. Xu, L. Bai, L. Cui, M. Li, Y. Wang, and E. R. Hancock. "HC-GAE: The Hierarchical Cluster-based Graph Auto-Encoder for Graph Representation Learning". In: *arXiv preprint arXiv:2405.14742* (2024) (cit. on p. 16).
- [66] M. Gao, S. Liu, Y. Qi, X. Guo, and X. Shang. "GAE-LGA: integration of multi-omics data with graph autoencoders to identify lncRNA–PCG associations". In: *Briefings in Bioinformatics* 23.6 (2022) (cit. on p. 16).
- [67] G. Jin, M. Wang, J. Zhang, H. Sha, and J. Huang. "STGNN-TTE: Travel time estimation via spatial–temporal graph neural network". In: *Future Generation Computer Systems* 126 (2022), pp. 70–81 (cit. on p. 16).
- [68] J. Yin, A. Qiu, L. Fang, N. Wang, C. Dong, and S. Ge. "STGNN: A Novel Spatial-Temporal Graph Neural Network for Predicting Complicated Business Process Performance under Multi-Event Parallelism". In: *Expert Systems with Applications* (2025), p. 128391 (cit. on p. 16).
- [69] W. Jiang. "Graph-based deep learning for communication networks: A survey". In: *Computer Communications* 185 (2021-03), pp. 40–54. ISSN: 0140-3664. doi: 10.1016/j.comcom.2021.12.015 (cit. on p. 17).

-
- [70] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio. “Unveiling the potential of Graph Neural Networks for network modeling and optimization in SDN”. In: *Proceedings of the 2019 ACM Symposium on SDN Research. SOSR '19*. San Jose, CA, USA: Association for Computing Machinery, 2019, pp. 140–151. ISBN: 9781450367103. DOI: 10.1145/3314148.3314357 (cit. on p. 17).
 - [71] H. Wei, Y. Zhao, and K. Xu. “G-Routing: graph neural networks-based flexible online routing”. In: *IEEE Network* 37.4 (2023), pp. 90–96 (cit. on p. 17).
 - [72] M. Ding, Y. Guo, Z. Huang, B. Lin, and H. Luo. “GROM: A generalized routing optimization method with graph neural network and deep reinforcement learning”. In: *J. Netw. Comput. Appl.* 229.C (2024-09). ISSN: 1084-8045. DOI: 10.1016/j.jnca.2024.103927 (cit. on p. 17).
 - [73] F. Zhou, X. Liu, R. Yu, and G. Xue. *Traffic Engineering in Large-scale Networks with Generalizable Graph Neural Networks*. 2025. arXiv: 2503.24203 [cs.NI] (cit. on pp. 17, 18).
 - [74] W. Jiang, H. Han, Y. Zhang, J. Wang, M. He, W. Gu, J. Mu, and X. Cheng. “Graph neural networks for routing optimization: Challenges and opportunities”. In: *Sustainability* 16.21 (2024), p. 9239 (cit. on p. 18).
 - [75] L. Luo, W. Ren, H. Huang, and F. Wang. “A Survey on Privacy Attacks and Defenses in Graph Neural Networks”. In: *Information Technology and Control* 53.4 (2024), pp. 1251–1278 (cit. on pp. 19, 20).
 - [76] J. Wei, L. Yaxin, X. Han, W. Yiqi, J. Shuiwang, and A. Charu. “Adversarial attacks and defenses on graphs: A review, a tool, and empirical studies”. In: *arXiv preprint arXiv:2003.00653* 653 (2020) (cit. on pp. 19, 20).
 - [77] D. Galli, A. Venturi, D. Stabili, M. Andreolini, and M. Marchetti. “Defending Network Intrusion Detection Systems Based on Graph Neural Networks Against Structural Adversarial Attacks”. In: *2025 23rd International Symposium on Network Computing and Applications (NCA)*. 2025, pp. 219–228. DOI: 10.1109/NCA67271.2025.00043 (cit. on pp. 19, 20).
 - [78] D. Zügner, A. Akbarnejad, and S. Günnemann. “Adversarial Attacks on Neural Networks for Graph Data”. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '18*. London, United Kingdom: Association for Computing Machinery, 2018, pp. 2847–2856. ISBN: 9781450355520. DOI: 10.1145/3219819.3220078 (cit. on p. 20).
 - [79] M. Aburidi and R. Marcia. “Topological adversarial attacks on graph neural networks via projected meta learning”. In: (2024) (cit. on p. 20).
 - [80] J. Chen, Y. Wu, X. Xu, Y. Chen, H. Zheng, and Q. Xuan. “Fast gradient attack on network embedding”. In: *arXiv preprint arXiv:1809.02797* (2018) (cit. on p. 20).

- [81] H. Dai, H. Li, T. Tian, X. Huang, L. Wang, J. Zhu, and L. Song. “Adversarial Attack on Graph Structured Data”. In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy and A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018-10–15 Jul, pp. 1115–1124. URL: <https://proceedings.mlr.press/v80/dai18b.html> (cit. on p. 20).
- [82] K. Xu, H. Chen, S. Liu, P.-Y. Chen, T.-W. Weng, M. Hong, and X. Lin. “Topology attack and defense for graph neural networks: An optimization perspective”. In: *arXiv preprint arXiv:1906.04214* (2019) (cit. on p. 20).
- [83] Y. Sun, S. Wang, X. Tang, T.-Y. Hsieh, and V. Honavar. “Adversarial Attacks on Graph Neural Networks via Node Injections: A Hierarchical Reinforcement Learning Approach”. In: *Proceedings of The Web Conference 2020*. WWW ’20. Taipei, Taiwan: Association for Computing Machinery, 2020, pp. 673–683. ISBN: 9781450370233. DOI: 10.1145/3366423.3380149 (cit. on p. 20).
- [84] Y. Ma, S. Wang, T. Derr, L. Wu, and J. Tang. “Attacking graph convolutional networks via rewiring”. In: *arXiv preprint arXiv:1906.03750* (2019) (cit. on p. 20).
- [85] H. Wu, C. Wang, Y. Tyshetskiy, A. Docherty, K. Lu, and L. Zhu. “Adversarial examples on graph data: Deep insights into attack and defense”. In: *arXiv preprint arXiv:1903.01610* (2019) (cit. on p. 20).
- [86] M. Aburidi and R. Marcia. “Defending Graph Neural Networks Against Adversarial attacks via Symmetric Matrix Factorization”. In: *2025 Conference on Artificial Intelligence x Multimedia (AIXMM)*. 2025, pp. 55–61. DOI: 10.1109/AIXMM62960.2025.00016 (cit. on p. 21).
- [87] L. Roy, P. Bhandari, S. Yadav, A. Akula, A. S. Reddy, and A. Lathigara. “Protecting Against Adversarial Attacks in Graph Neural Networks: Vulnerability Assessment and Parallel Defense Strategies”. In: *2025 World Skills Conference on Universal Data Analytics and Sciences (WorldSUAS)*. 2025, pp. 1–6. DOI: 10.1109/WorldSUAS66815.2025.11199215 (cit. on p. 21).
- [88] D. R. Pedro Amaral. “Multi-Agent Deep Reinforcement Learning for Adaptive Routing: A Study of Critic Design and Graph Neural Network State Encoding”. In: (2021) (cit. on pp. 23, 25).



2026

A Very Long and Impressive Thesis Title With a Forced Line Break Miguel Chen